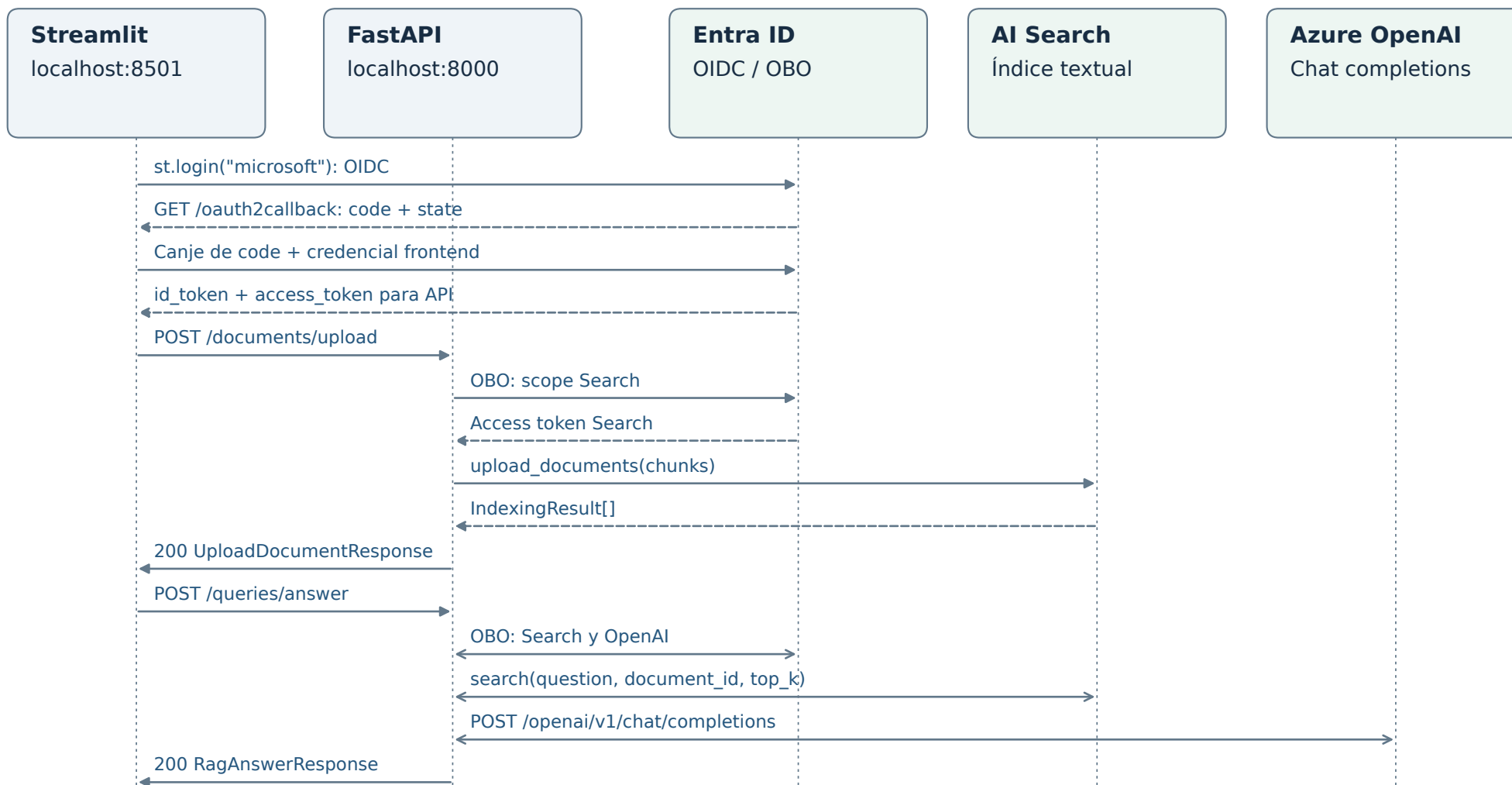


Secuencia HTTP y llamadas a Azure

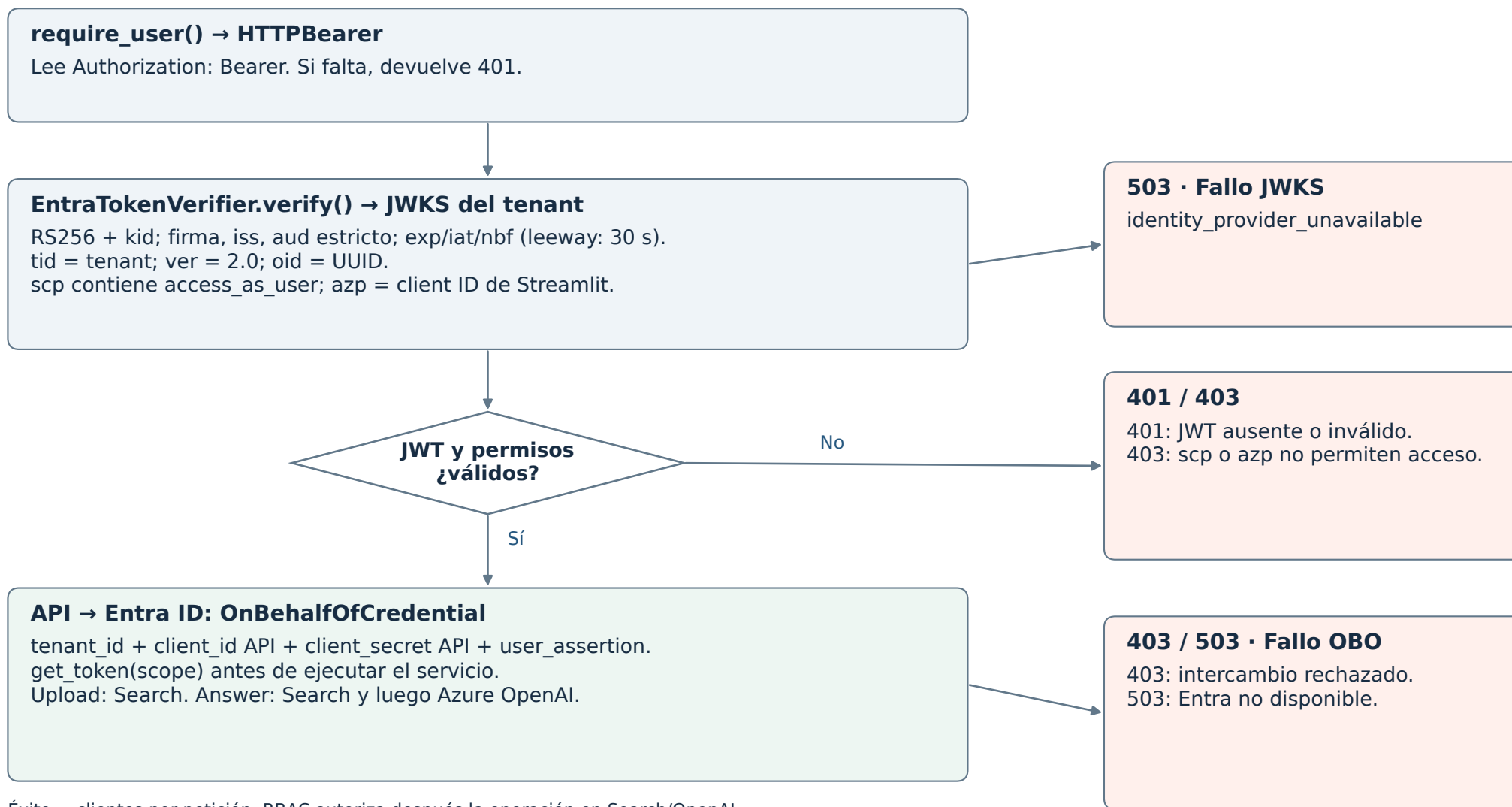
Cliente HTTP: proceso Streamlit. FastAPI coordina Azure con la identidad del usuario.



Rutas UI → API bajo /api/v1; Authorization: Bearer <token API>. La flecha doble resume request + response.

Autenticación: JWT y acceso delegado

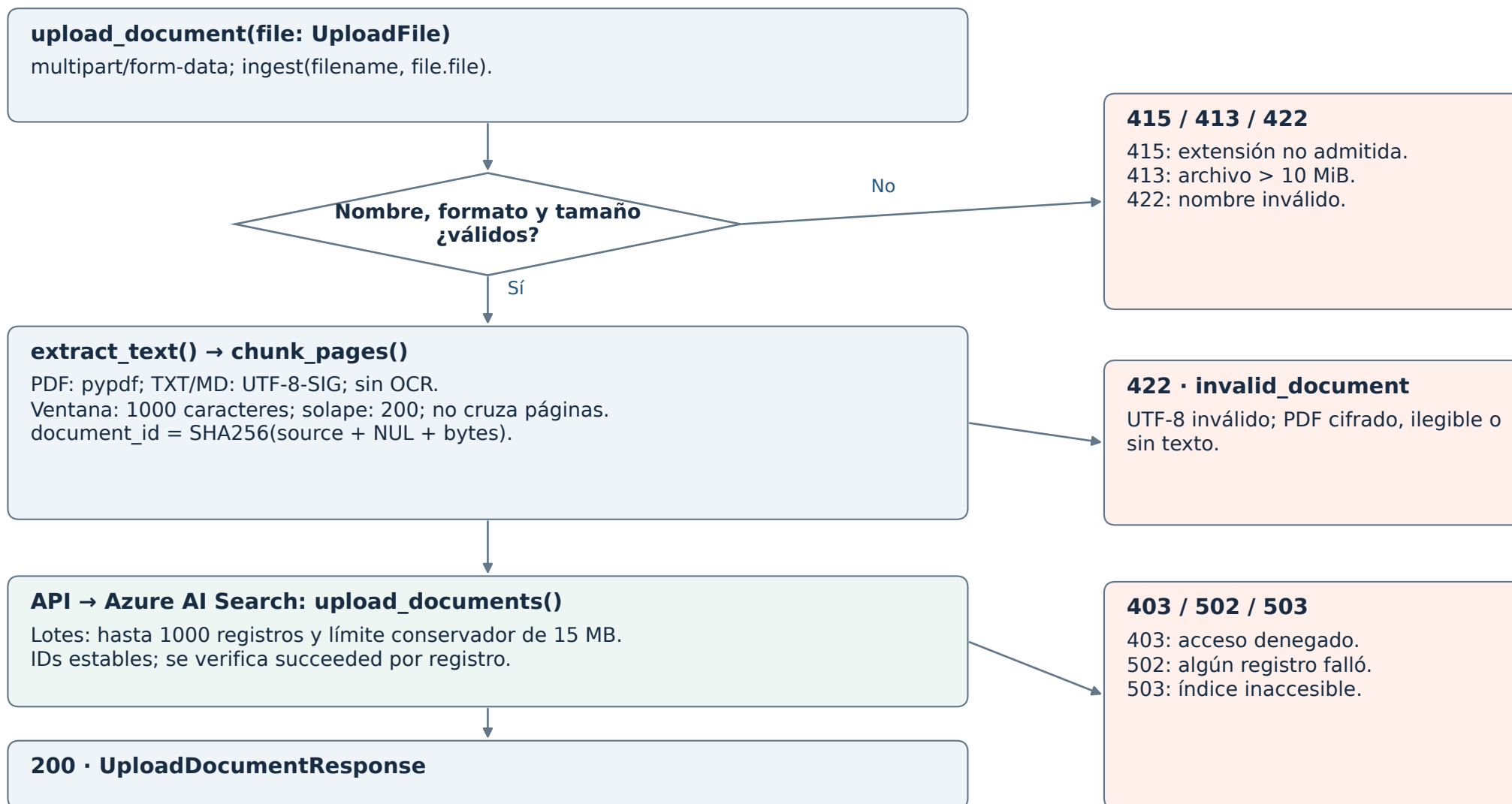
Pipeline compartido por documentos y consultas; el secreto de FastAPI autentica el intercambio OBO.



Éxito → clientes por petición. RBAC autoriza después la operación en Search/OpenAI.

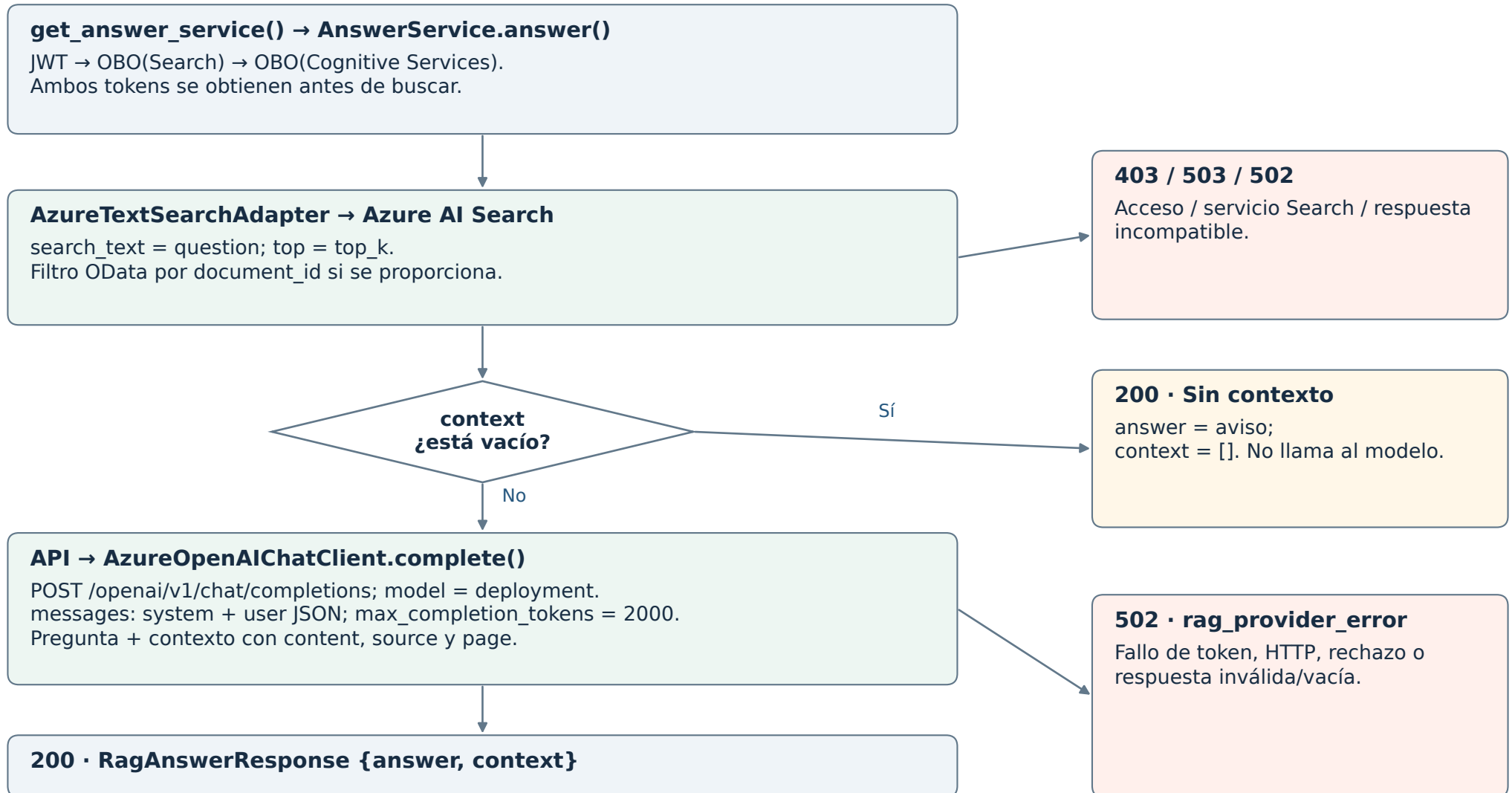
Ingestión: del archivo al índice textual

POST /api/v1/documents/upload · Dependencias: JWT → OBO(Search) → FileIngestionService.



Consulta RAG: búsqueda, contexto y modelo

POST /api/v1/queries/answer · Pydantic: question, document_id opcional y top_k.



Scopes, contratos HTTP y límites

Scopes distintos por destinatario. Consentimiento de Entra y roles de datos son controles separados.

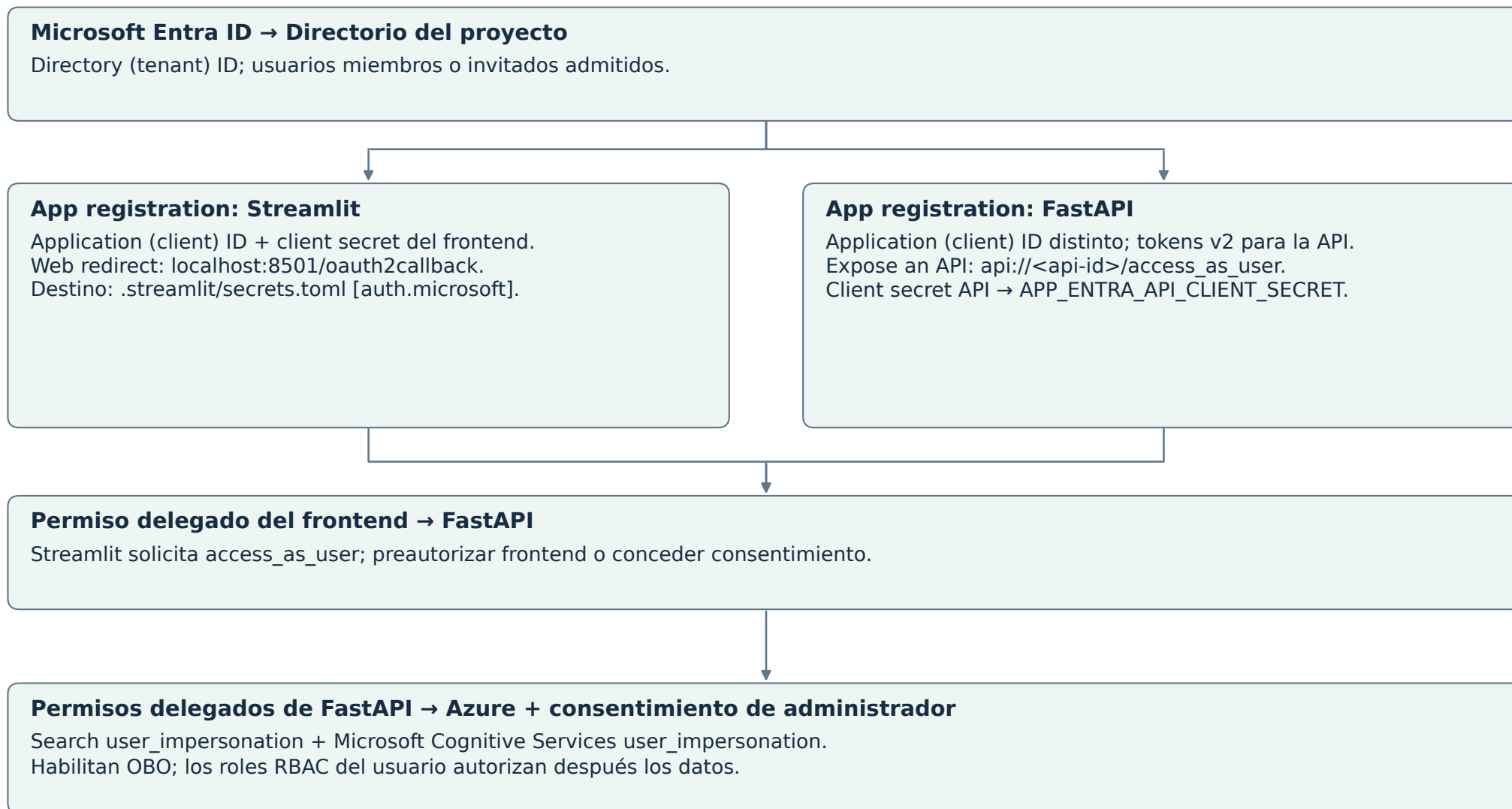
Destino del token	Scope solicitado	Control / rol de datos
FastAPI	api://<api-client-id>/access_as_user	aud = client ID API; scp incluye access_as_user
Azure AI Search	https://search.azure.com/.default	Search Index Data Contributor (carga y consulta de la home)
Azure OpenAI	https://cognitiveservices.azure.com/.default	Cognitive Services OpenAI User

CONTRATOS · Prefijo /api/v1 · Respuestas de éxito: HTTP 200

Método y ruta	Entrada / límites	Salida
POST /documents/upload	file: PDF con texto / TXT / MD; ≤ 10 MiB	document_id, source, indexed_chunks, warnings
POST /queries/answer	question: 1-4000 caracteres; top_k: 1-20 (UI: 5); document_id opcional	answer: string; context: SearchHit[]
POST /documents/chunks	chunks: 1-1000; embeddings precalculados	indexed_chunks
POST /queries/search	embedding precalculado; top_k: 1-50; document_id opcional; índice vectorial	matches: SearchHit[]
GET /health	Público; no prueba conexiones con Azure	Estado y configuración

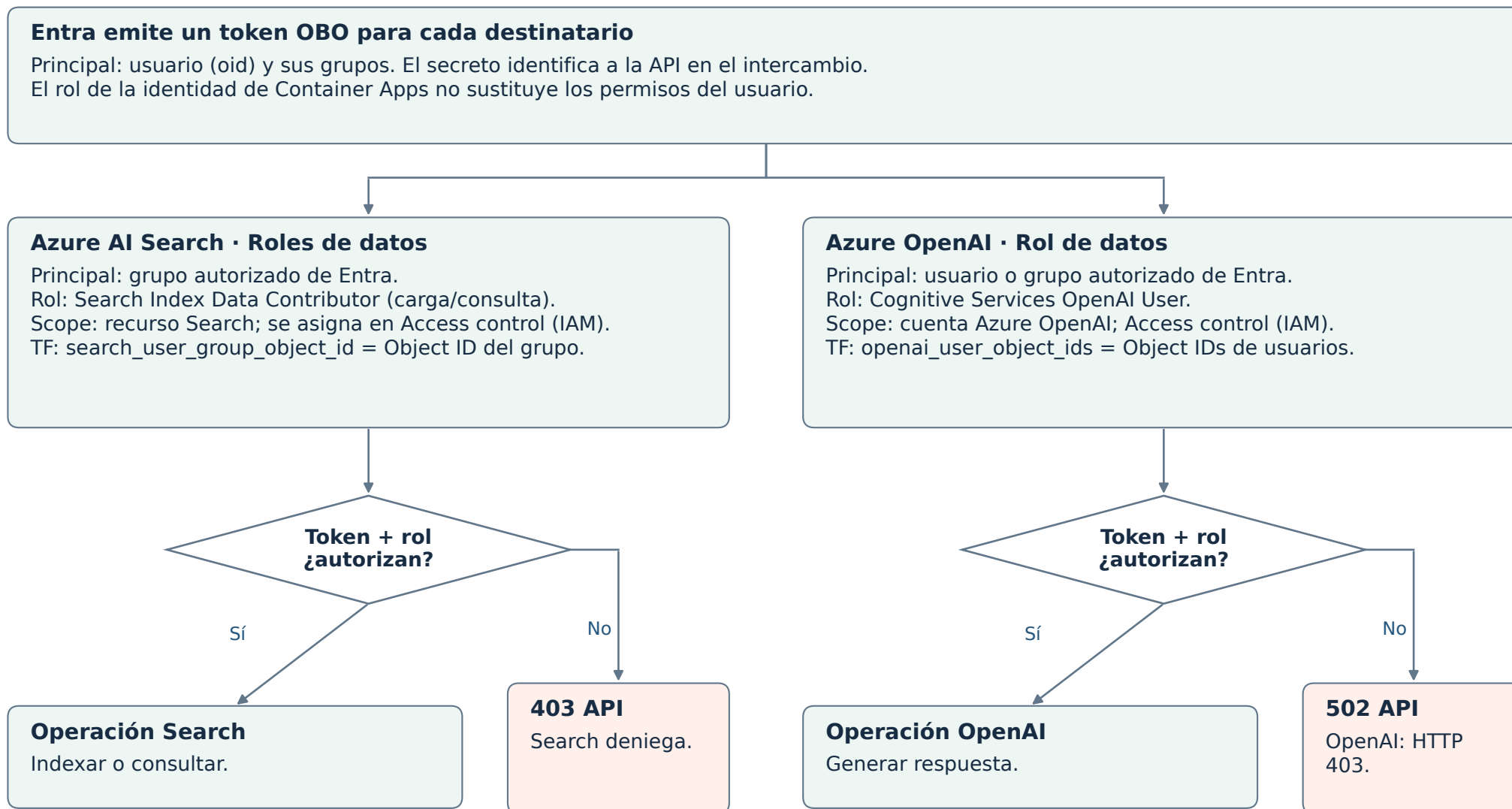
Entra ID: registros y consentimiento

Configuración previa: dos registros distintos, permisos delegados y consentimiento.



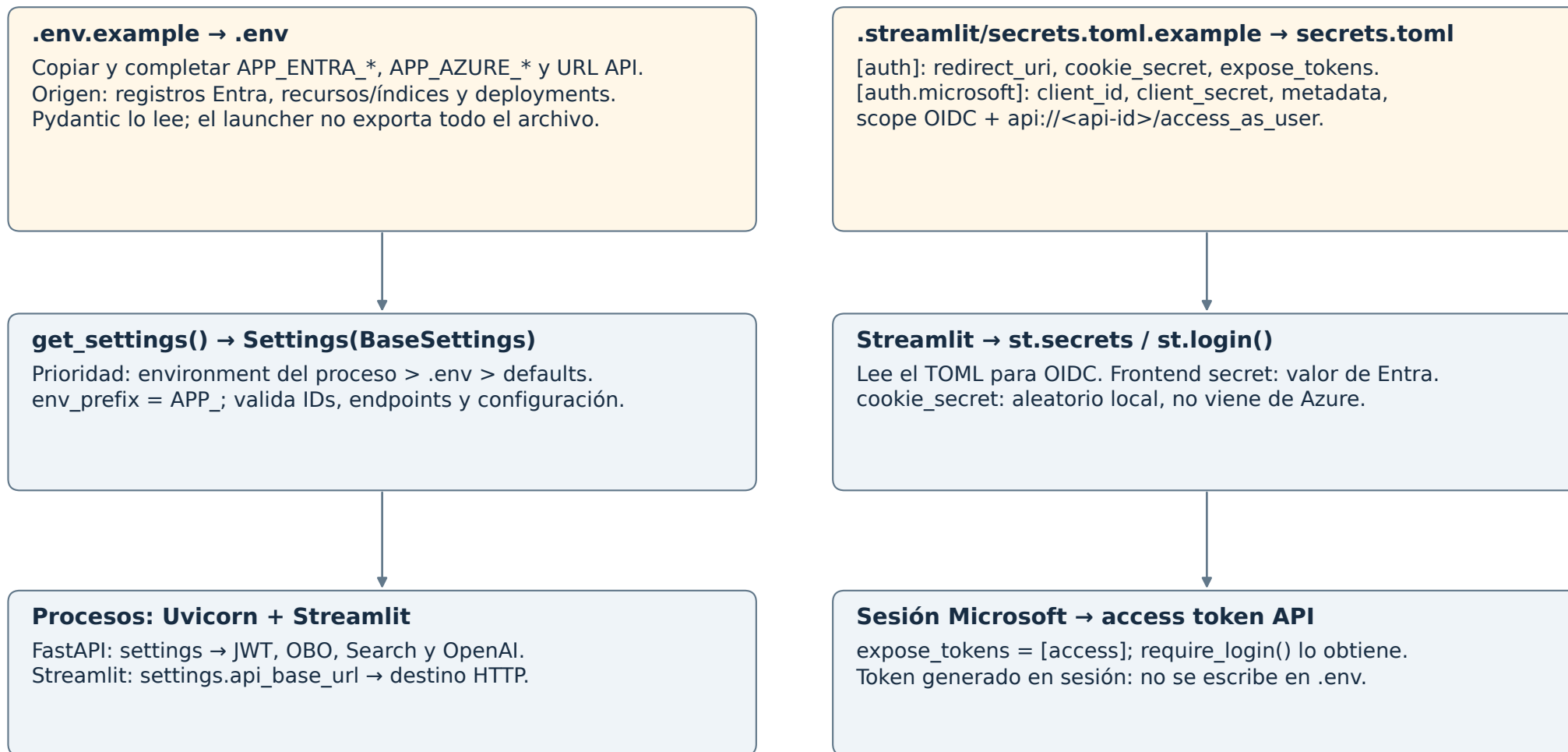
RBAC: autorización sobre los recursos

RBAC = principal + rol + scope del recurso. Se evalúa en el servicio Azure.



Configuración: archivos y procesos locales

Archivos en la raíz del proyecto; Settings y st.secrets leen configuraciones distintas.



run_local.sh exporta APP_API_BASE_URL desde host/puerto o environment existente; prevalece sobre .env.

De dónde sale cada variable

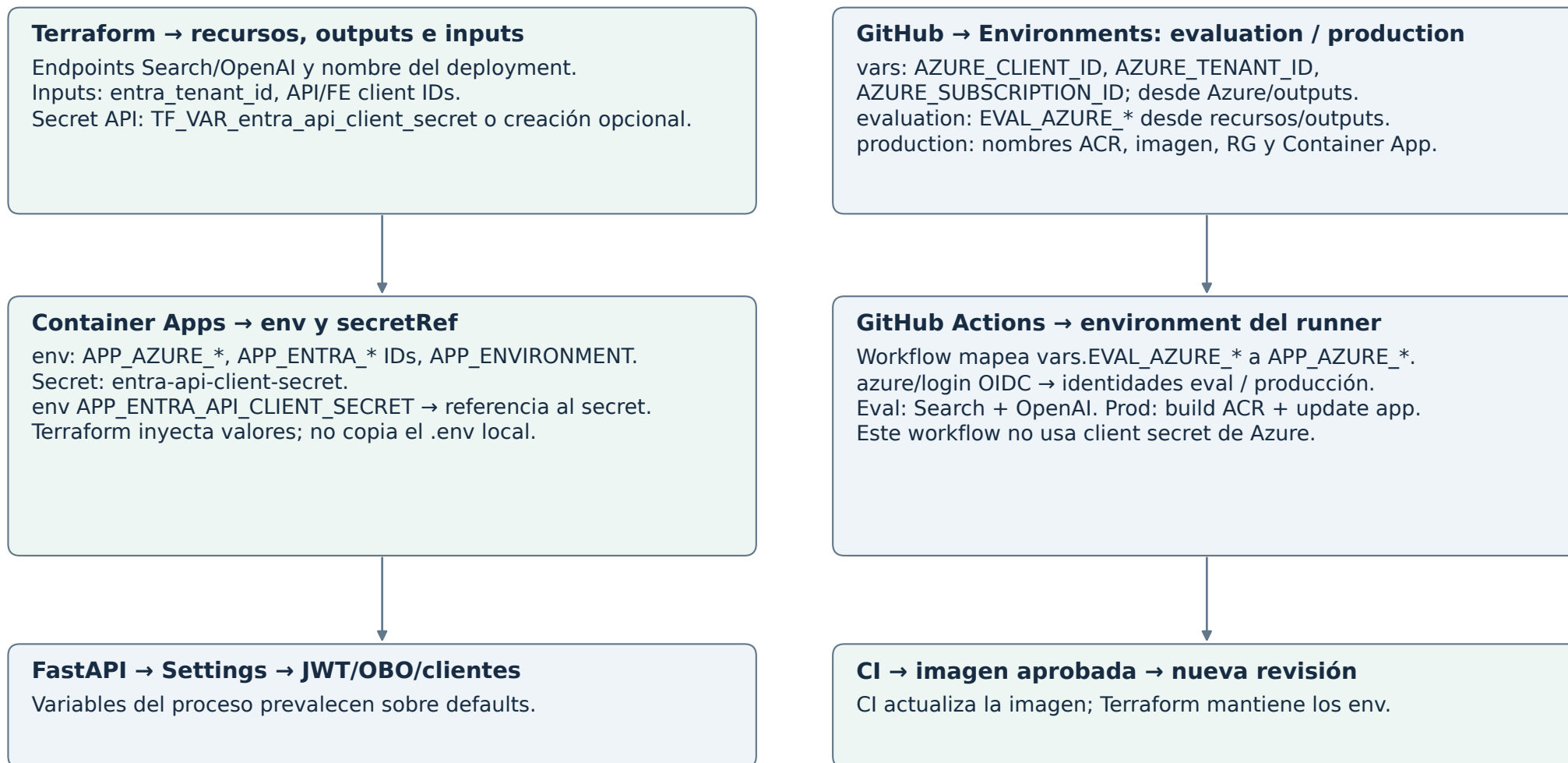
Tabla de origen → variable → consumidor. Cada nombre de la primera columna lleva APP_.

Variable en .env (prefijo APP_)	Dónde obtener el valor	Dónde se usa
ENTRA_TENANT_ID	Entra ID: Overview → Directory (tenant) ID	JWT issuer/tid + OBO; tenant también en metadata OIDC
ENTRA_API_CLIENT_ID	App registration FastAPI → Application (client) ID	JWT aud; OBO client_id; scope API en el TOML
ENTRA_FRONTEND_CLIENT_ID	App registration Streamlit → Application (client) ID	JWT azp; mismo ID en [auth.microsoft].client_id
ENTRA_API_CLIENT_SECRET	Registro FastAPI → Certificates & secrets → VALUE	OBO; valor del secreto, no Secret ID
AZURE_SEARCH_ENDPOINT	Recurso Search: Overview / output search_service_endpoint	SearchClient.endpoint
AZURE_SEARCH_TEXT_INDEX_NAME	Search → Indexes; Terraform crea índice textual	SearchClient.index_name
AZURE_OPENAI_ENDPOINT	Endpoint del recurso / output azure_openai_endpoint	Base HTTPS del cliente chat
AZURE_OPENAI_CHAT_DEPLOYMENT	Deployment de modelo / output azure_openai_chat_deployment_name	Campo model de chat; no el ID del recurso
API_BASE_URL	Local: launcher host/puerto. Azure: output container_app_url	Streamlit → destino FastAPI

TOML: client_secret = VALUE del secreto del frontend; cookie_secret = aleatorio generado. API y frontend usan secretos distintos.

Configuración: Azure y GitHub Actions

Terraform configura la API; GitHub Environments configura los jobs. Son ámbitos distintos.



Key Vault está preparado, sin conexión actual como fuente de env. Streamlit configura su TOML y URL API en su hosting.